

FastWAM + CoT DiT：基于思维链的世界-动作模型实现计划

初稿

上海创智博士夏令营三人组队项目

2026 年 5 月 19 日

摘要

本文档给出在 FastWAM 代码库上实现 WAM-CoT 的初步技术计划。核心思想是在已有 Video DiT 与 Action DiT 的 MoT 框架中新增一个 CoT DiT 分支：其输入为由 Qwen3-VL-2B 提取的 VLM tok，结构参考 Motus 的 Understanding Expert，并通过统一的 MoT attention mask 与 Video DiT、Action DiT 交互。该设计希望在保留 FastWAM 高效动作预测能力的同时，引入来自 VLM 的场景理解、空间关系与任务语义先验，为长视野、遮挡和多阶段操作任务提供内部物理思维链表征。

1 项目目标与背景

课题目标是研究基于思维链机制的世界-动作模型（WAM-CoT）是否能够提升机器臂在复杂物理交互任务中的策略性能，尤其关注遮挡、长视野、多阶段堆叠等容易出现误差累积和策略崩溃的任务。本文选择“内部物理思维链”路线：不把 CoT 仅作为外部文本规划，而是在 WAM 内部加入可与视频 token 和动作 token 共同推理的 latent reasoning token。

FastWAM 当前以 Wan2.2 为 Video DiT backbone，额外构造 Action DiT 分支，并通过 Mixture-of-Transformers (MoT) 让 action token 与输入帧的 video VAE latent 交互。训练时 Video DiT 预测未来视频噪声，Action DiT 预测未来动作噪声；推理时可跳过未来视频生成，仅基于首帧和语言/状态条件预测动作，从而提高策略执行效率。

Motus 同样基于 Wan2.2，但进一步引入 Qwen3-VL-2B 和 Understanding Expert，使 VLM 的语义理解 token 参与 Tri-model Joint Attention。本文计划将这一思路迁移到 FastWAM，形成包含 Video DiT、Action DiT、CoT DiT 三个专家的 FastWAM+CoT DiT。

2 已有基础

2.1 FastWAM 的两专家 MoT

FastWAM 的核心专家包括：

- Video DiT：继承 Wan2.2 TI2V-5B 的视频生成结构。输入为 VAE latent，经 patchify 后形成 video token；其 hidden size 为 3072，30 层，24 个 attention head，每个 head 128 维。

- Action DiT: 与 Video DiT 层数、head 数、head 维度保持一致, hidden size 为 1024, FFN 维度为 4096。输入为动作序列, 经 action encoder 映射为 action token, 并用独立 flow timestep 做 AdaLN 调制。
- MoT: 每一层分别由各 expert 产生自己的 Q/K/V, 然后按 expert 顺序拼接, 在同一个 attention mask 下做 mixed self-attention, 最后再按 expert 切分输出并接回各自的 cross-attention、FFN 和 head。

当前 FastWAM 的基础 mask 逻辑为: Video DiT 内部使用 `first_frame_causal`, 即首帧 token 只能看首帧, 未来 video token 可以看首帧和未来 video token; Action DiT 内部 token 之间全连接; Action DiT 只能看首帧 video token。这对应“动作只依赖当前观测, 不偷看未来视频”的策略设定。

2.2 Motus 的三专家思想

公开 Motus 资料显示, 其 MoT 由三类专家组成: Video Generation Model (Wan2.2-5B)、Action Expert 和 Understanding Expert。三者通过 Tri-model Joint Attention 共享多头自注意力的信息流, 但各自保留独立的 normalization、FFN、输入/输出适配器和 timestep 处理逻辑。Motus 的设计动机是把视频生成的物理先验、动作建模的控制先验、VLM 的语义理解先验放在同一个联合生成框架内。

Motus 中 Action Expert 的公开实现要点如下:

- 输入编码: finetune 模式下分别编码 state token 和 action token, 再拼接; pretrain 模式下可只编码 action token。编码器使用 `mlp3x_silu`, 并加入固定一维正余弦位置编码。
- 主干结构: 层数与 Wan 一致。每个 block 包含 LayerNorm、AdaLN timestep 调制、FFN, 以及映射到 Wan attention head space 的 action Q/K/V 投影。
- MoT 交互: Action Expert 不直接复用 Wan 的 hidden size, 而是用参数 `wan_action_qkv` 将 action hidden 映射到 Wan 的 `num_heads × head_dim` 空间, 参与 joint attention 后再通过 `wan_action_o` 投回 action hidden size。
- 输出头: Action Decoder 使用 timestep 调制后的 LayerNorm 和 MLP 输出动作噪声/速度场。

Motus 中 Understanding Expert 的公开实现要点如下:

- 输入来自 VLM 最后一层 hidden states。公开配置中 VLM 为 Qwen3-VL-2B-Instruct, VLM hidden size 为 2048。
- VLM 输出先经过 `mlp3x_silu` adapter, 从 2048 维投影到 Understanding Expert hidden size, 公开 RobotWin 配置中为 512。
- Understanding Expert 与 Action Expert 类似, 也包含与 Wan 层数对齐的 Transformer blocks, 并用 `wan_und_qkv` 映射到 Wan head space 参加 Tri-model Joint Attention。
- 与 Action Expert 不同, Understanding Expert 通常没有动作输出 decoder, 也不需要 registers; 其作用是更新并传递 VLM 语义 token, 让 video/action 分支在每层 attention 中读取语义理解表征。

3 VLM Tok 的获取方式

为严格对齐 Motus，建议将 VLM tok 的产生过程写成独立模块，例如 `VLMTokenExtractor`，并在数据预处理或训练 forward 中复用同一逻辑。具体步骤如下：

1. 构造 Qwen3-VL 对话输入。输入包括当前 RGB 图像和任务语言指令，格式为一条 user message：图像在前，文本指令在后。
2. 调用 Qwen processor 的 `apply_chat_template(..., add_generation_prompt=True)`，再通过 `process_vision_info` 处理图像，最终得到 `input_ids`、`attention_mask`、`pixel_values`、`image_grid_thw`。
3. 使用 Qwen3-VL 的 embedding 层生成 `inputs_embeds`，调用 VLM 的 image encoder 得到 image features，并用 placeholder mask 将 image features 写回对应 image token 位置。
4. 根据 `input_ids`、`image_grid_thw` 和 `attention_mask` 调用 Qwen3-VL 的 RoPE index 逻辑生成 `position_ids`。
5. 冻结 VLM 参数，用 `output_hidden_states=True` 前向 Qwen3-VL language model，取 `hidden_states[-1]` 作为 VLM last-layer token。
6. 通过 CoT/Understanding adapter 将 VLM hidden states 投影到 CoT DiT hidden size，得到本文定义的 VLM tok。

形式化地，设 Qwen3-VL 最后一层 hidden states 为

$$H^{\text{vlm}} = \text{Qwen3VL}_{\text{frozen}}(I_0, \ell)_{\text{last}} \in \mathbb{R}^{B \times L_u \times d_{\text{vlm}}},$$

其中 I_0 是当前观测图像， ℓ 是语言指令。VLM tok 为

$$U_0 = \phi_{\text{adapter}}(H^{\text{vlm}}) \in \mathbb{R}^{B \times L_u \times d_u},$$

其中 ϕ_{adapter} 建议使用 Motus 一致的 `mlp3x_silu`。

如果本项目内部称“Qwen-3-2B”，实现上建议以 Motus 配置中的 `Qwen3-VL-2B-Instruct` 为准；若实际使用纯文本 Qwen3-2B，则无法提供图像 token，和 Motus 的 VLM tok 定义不完全一致，需要另行改写输入语义。

4 CoT DiT 的网络架构设计

4.1 设计原则

CoT DiT 应满足三个约束：

- 与 FastWAM 的 MoT 接口兼容：必须提供 `tokens`、`freqs`、`t_mod` 或等价字段，并包含与 Video/Action 对齐的 `blocks`、`num_heads`、`attn_head_dim`。
- 与 Motus Understanding Expert 对齐：输入来自 Qwen3-VL last hidden states，经 adapter 后参与每层 joint attention；不直接预测动作。

- 与 FastWAM 的计算预算匹配：CoT DiT 不应显著破坏 FastWAM 的推理效率，因此 hidden size 和 token 数要可控。

4.2 推荐配置

本文建议第一版使用如下配置：

模块	推荐值	说明
VLM backbone	Qwen3-VL-2B-Instruct	与 Motus 对齐，训练时冻结
VLM input dim	2048	Qwen3-VL-2B last hidden size
CoT hidden size d_u	512	参考 Motus Understanding Expert，节省显存
CoT FFN dim	2048	4 倍 hidden size
层数	30	与 Wan2.2 / Action DiT 对齐
attention heads	24	与 MoT mixed attention 对齐
head dim	128	与 Video/Action attention head space 对齐
adapter	mlp3x_silu	2048 $\rightarrow$ 512
输出 head	无或 lightweight probe	默认不直接监督输出

CoT DiT block 可采用：

$$\begin{aligned}
\tilde{U}^{(l)} &= \text{LN}_1(U^{(l)}), \\
Q_u, K_u, V_u &= \text{Proj}^{u \rightarrow \text{WanHead}}(\tilde{U}^{(l)}), \\
Y_v, Y_a, Y_u &= \text{MoTAttn}([Q_v, Q_a, Q_u], [K_v, K_a, K_u], [V_v, V_a, V_u]; M), \\
U^{(l+\frac{1}{2})} &= U^{(l)} + \text{Proj}^{\text{WanHead} \rightarrow u}(Y_u), \\
U^{(l+1)} &= U^{(l+\frac{1}{2})} + \text{FFN}(\text{LN}_2(U^{(l+\frac{1}{2})})).
\end{aligned}$$

与 Action DiT 的区别是：CoT DiT 不需要 action noise scheduler，也不需要输出动作维度；如果希望让 CoT token 也具备 denoising 式 latent reasoning，可加入一个固定或可学习的 CoT timestep embedding，但第一版建议不引入额外 diffusion loss，先把 CoT DiT 作为语义-物理中间表征分支。

5 Attention Mask 设计与实现

5.1 符号

本文把 token 分组为：

- u : VLM tok / CoT token;
- f_0 : 首帧 video latent token;
- f_1, f_h : 未来 video latent token, 其中 f_1 可表示第一组未来 token, f_h 表示后续 horizon token;
- a_1, a_h : 未来动作 token。

在实现中无需按 f_1/f_h 单独存储，只需要知道每帧 video token 数 `video_tokens_per_frame`，即可切出 f_0 和未来 video token。

5.2 训练 mask

训练时的目标 mask 为：

Query \ Key	VLM tok	f_0	f_1	f_h	a_1	a_h
VLM tok	✓	✓				
f_0	✓	✓				
f_1	✓	✓	✓	✓		
f_h	✓	✓	✓	✓		
a_1	✓	✓			✓	✓
a_h	✓	✓			✓	✓

该 mask 的含义是：

- VLM tok 只读取自身和当前首帧，避免被未来视频或动作标签污染；
- Video token 可读取 VLM tok，从语义理解中获得物体、空间关系和任务目标信息；
- 未来 video token 仍维持 FastWAM 原有 video-to-video 可见性；
- Action token 可读取 VLM tok 和首帧 token，但不可读取未来 video token，避免训练时从未来视觉答案中泄漏动作信息；
- Action token 内部全连接，维持 action chunk 的整体预测。

5.3 推理 mask

推理只保留当前观测和 action token，因此目标 mask 为：

Query \ Key	VLM tok	f_0	a_1	a_h
VLM tok	✓	✓		
f_0	✓	✓		
a_1	✓	✓	✓	✓
a_h	✓	✓	✓	✓

这与 FastWAM 当前高效动作推理保持一致：不需要在测试时生成未来视频，只需 prefill u 和 f_0 ，再 denoise action token。

5.4 代码实现草案

FastWAM 当前的 `_build_mot_attention_mask(video_seq_len, action_seq_len, video_tokens_per_device)` 返回 $[S, S]$ bool mask。加入 CoT DiT 后建议改为：

```
def _build_cot_mot_attention_mask(
    cot_seq_len: int,
    video_seq_len: int,
```

```

    action_seq_len: int,
    video_tokens_per_frame: int,
    device: torch.device,
) -> torch.Tensor:
    total = cot_seq_len + video_seq_len + action_seq_len
    mask = torch.zeros((total, total), dtype=torch.bool, device=device)

    u0, u1 = 0, cot_seq_len
    v0, v1 = u1, u1 + video_seq_len
    a0, a1 = v1, total
    first = min(video_tokens_per_frame, video_seq_len)

    # cot -> cot + f0
    mask[u0:u1, u0:u1] = True
    mask[u0:u1, v0:v0 + first] = True

    # video -> cot + original video mask
    mask[v0:v1, u0:u1] = True
    mask[v0:v1, v0:v1] = self.video_expert.build_video_to_video_mask(...)

    # action -> cot + f0 + action
    mask[a0:a1, u0:u1] = True
    mask[a0:a1, v0:v0 + first] = True
    mask[a0:a1, a0:a1] = True
    return mask

```

MoT 的 expert 顺序应固定为 {"cot": cot_expert, "video": video_expert, "action": action_expert}, 从而 mask 的行列顺序与 token 拼接顺序一致。若为了最小改动保留 video, action 顺序, 也可以将 CoT 追加到末尾, 但 mask 构造必须同步调整。

6 FastWAM+CoT DiT 完整 Pipeline

6.1 训练流程

1. 从样本中读取多相机/单相机视频、动作序列、语言指令和 proprioception。
2. 使用 Video VAE 编码视频, 得到 latent video; 首帧作为条件, 未来帧加入 flow-matching 噪声。
3. 对动作序列加入 action scheduler 噪声, 得到 noisy action token。
4. 使用 Qwen3-VL-2B 提取当前图像和语言指令的 last hidden states, 经过 adapter 得到 VLM tok。

5. 分别执行 `video_expert.pre_dit`、`action_expert.pre_dit`、`cot_expert.pre_dit`，得到三类 token、RoPE 频率和调制参数。
6. 构建训练 attention mask，将 VLM tok、video token、action token 拼接后送入三专家 MoT。
7. Video DiT head 输出 video velocity/noise prediction；Action DiT head 输出 action velocity/noise prediction；CoT DiT 默认不设主损失。
8. 总损失第一版保持

$$\mathcal{L} = \lambda_v \mathcal{L}_{video} + \lambda_a \mathcal{L}_{action}.$$

后续可加入辅助损失，如 VLM token dropout consistency、阶段成功预测 probe 或 object relation probe。

6.2 推理流程

1. 输入当前观测图像、语言指令和 proprioception。
2. VAE 编码首帧得到 f_0 ；Qwen3-VL 提取 VLM tok 得到 u 。
3. 初始化 action latent 为高斯噪声。
4. 每个 denoising step 中，构建推理 mask，让 action token 读取 u 、 f_0 和 action token 自身。
5. Action DiT 输出动作噪声/速度场，scheduler 更新 action latent。
6. 完成所有 step 后反归一化动作 chunk，并交给仿真环境执行。

7 实验与消融计划占位

后续实验建议至少包含：

- Baseline：原始 FastWAM；
- Ours：FastWAM+CoT DiT；
- 去除 CoT：保留参数量相近的空 expert 或关闭 VLM tok；
- 打乱 VLM 输入：打乱语言或替换图像，验证 CoT token 是否真实参与动作生成；
- mask 消融：允许 action 看未来 video token、禁止 action 看 VLM tok、禁止 video 看 VLM tok；
- 任务维度：遮挡抓取、多阶段堆叠、长视野搬运等。

主要指标包括任务成功率、阶段成功率、平均执行步数、失败类型统计、推理时延、显存占用和 action chunk 稳定性。[待补充：补充具体任务、数据规模、训练超参和评估脚本路径。]

8 实现里程碑

1. M1: 复现 FastWAM baseline 训练/推理, 确认 RoboTwin 或 LIBERO 指标可跑通。
2. M2: 加入 Qwen3-VL VLM tok 提取脚本, 支持离线缓存, 降低训练时显存和吞吐压力。
3. M3: 实现 CoT DiT / Understanding Expert, 并接入三专家 MoT。
4. M4: 实现训练和推理 attention mask, 完成 shape test 和 overfit 小样本测试。
5. M5: 完整训练 FastWAM+CoT DiT, 进行消融实验和失败案例分析。

9 当前风险与待确认事项

- 本机当前不可访问用户提供的私有 Motus 路径 /pfs/pfs-iQ14no/yuhao/Motus, 本文 Motus 细节基于公开论文、README 和公开仓库代码整理。若私有版本有改动, 需要以后按源码校正。
- CoT DiT 引入 Qwen3-VL, 会显著增加显存占用。第一版应优先支持 VLM tok 离线缓存, 训练时冻结 VLM。
- VLM token 长度可能较大, 需要考虑截断、只保留 image/text 有效 token、或使用 pooling/query compression。
- 如果 Action token 读取 VLM tok 后出现训练不稳定, 可先冻结 CoT DiT 前若干 epoch, 仅训练 adapter 和 Action/Video 侧。

参考文献

- [1] Fast-WAM: Do World Action Models Need Test-time Future Imagination? arXiv:2603.16666.
- [2] Hongzhe Bi et al. Motus: A Unified Latent Action World Model. arXiv:2512.13030.
- [3] Wan-AI. Wan2.2-TI2V-5B model family.
- [4] Qwen Team. Qwen3-VL-2B-Instruct.
- [5] CoT-VLA: Visual Chain-of-Thought Reasoning for Vision-Language-Action Models. arXiv:2503.22020.